

Based on the provided sources, the interaction between **PHP** (the server-side processor) and **MySQL** (the database storage) follows a specific architectural flow that is a core topic in web development exams 1, 2.

Below are the key types of programs and logic patterns involving PHP and MySQL connection and interaction likely to appear in exams.

1. The Data Flow Architecture

Exams often ask how data travels from a user to a database. The sources define this pipeline as:

1. **Front-end:** A user interacts with an HTML form (e.g., entering a username/password) 3.
2. **HTTP Request:** The front-end sends this data to the **Backend Web Server** using methods like **POST** (for security) or **GET** (for parameters) 2, 4.
3. **PHP Processing:** The PHP script on the server receives the data (via `$_POST` or `$_GET`), processes it, and generates a **SQL Statement** 5, 6.
4. **Database Execution:** The server sends the SQL command to the **MySQL Database**, which performs the requested operation and returns the results 6, 7.

2. CRUD Operations in Programs

Programs involving **CRUD** (Create, Read, Update, Delete) are the "bread and butter" of database interaction 8.

A. Create (Insertion Program)

This involves taking user input from a PHP form and using an **INSERT** statement to add a new record to a table 8, 9.

- **Logic Example:**
- **PHP:** Use `$_POST["username"]` to grab form data 10.
- **SQL:** `INSERT INTO users (username, password) VALUES ('David_dev', 'insert_practice');` 11.
- **Exam Tip:** Remember that the **Auto Increment** keyword allows the database to generate unique IDs automatically, so you don't have to manually provide an ID in your **INSERT** statement 12, 13.

B. Read (Selection Program)

This is a "Read" operation using the **SELECT** statement to retrieve data to be displayed on a webpage 8, 14.

- **Logic Example:**
- **SQL:** `SELECT name, balance FROM users WHERE is_admin = true;` 15, 16.
- **PHP:** Use the `echo` command to render the retrieved database rows into the HTML document for the user to see 17, 18.

C. Update and Delete

- **Update:** Changing existing data, such as setting an `is_admin` flag to true for a specific user ID 19, 20.
- **Delete:** Removing a record. **Exam Warning:** Always use a WHERE clause with a specific **Primary Key** (like an ID) to avoid accidentally deleting every row in the table 21-23.

3. Form Handling Methods: GET vs. POST

A common exam question is the difference between these two methods when connecting PHP to database logic:

- **GET:** Appends data to the URL (e.g., `site.php?name=Mike`). It is useful for **bookmarking** and public data but **extremely insecure** for sensitive information 24-27.
- **POST:** Passes data securely in the request body without showing it in the URL. It is the **preferred method** for sensitive data like passwords or credit card numbers 4, 28, 29.

4. Advanced Logic: Joins and Relationships

For more advanced exams, you may need to write programs that query multiple related tables using **Joins** 30.

- **Inner Join:** Returns records only if there is a match in both tables (e.g., matching a user to their specific country code) 31, 32.
- **Left Join:** Returns all records from the first ("left") table, even if there is no match in the second table. This is used if you want to see all users, regardless of whether they have made any transactions 33, 34.

5. Security: Preventing SQL Injection

Exam questions often cover the danger of **SQL Injection**, where users inject malicious code (like `DROP TABLE`) into a form 35, 36.

- **Safe Practice:** Instead of using manual string concatenation to build queries, developers should use **variable interpolation** or modern libraries that handle **sanitization** automatically 37-39.